

AI-Assisted Software Development

Module 1: Introduction to aider

Mike Burton

What is aider?

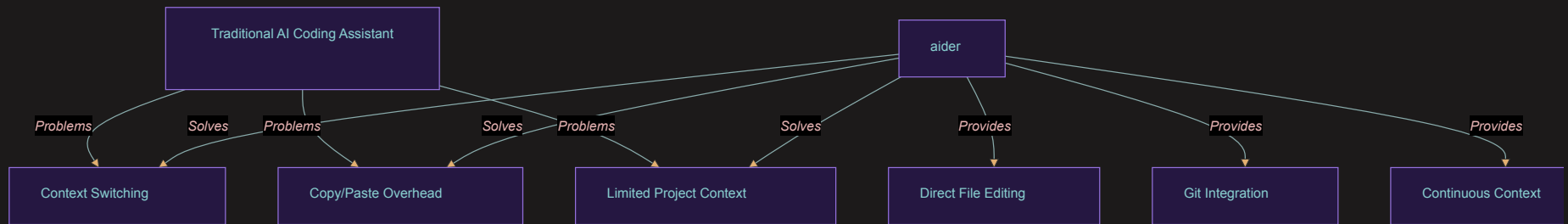
- An AI-powered coding assistant that acts as a pair programmer
- Uses OpenAI's GPT models to help write, edit, and refactor code
- Works directly in your terminal and with your local files
- Integrates with Git for version control

Why was aider created?

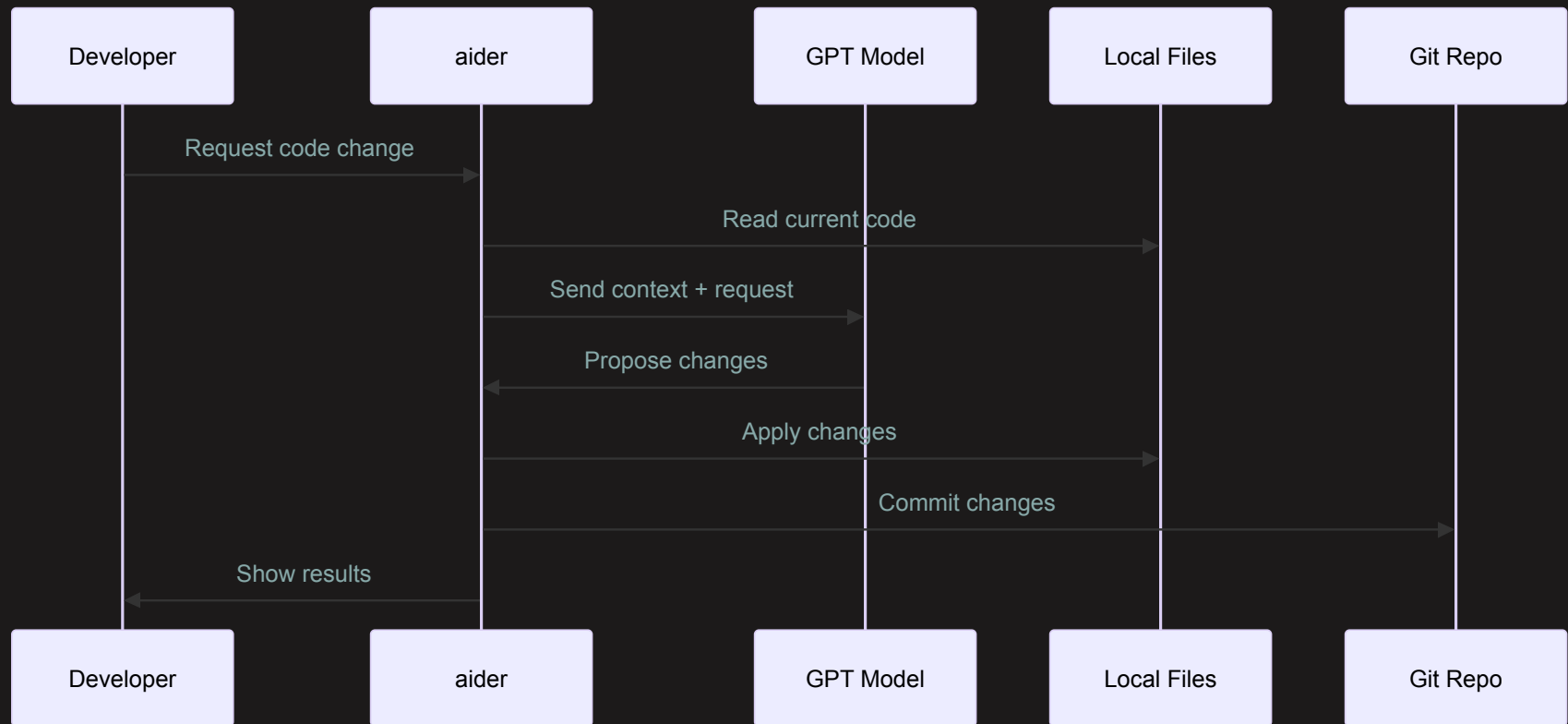
The Challenge:

- Traditional AI chat interfaces aren't optimized for coding
- Copying/pasting between chat and editor is inefficient
- Context switching breaks flow
- Hard to maintain project context

The Solution:



How aider Works



Key Features

1. Direct File Editing

- Modifies your code files in place
- No copy/paste needed
- Maintains file structure

2. Git Integration

- Automatic commits
- Clear change history
- Safe experimentation

3. Natural Language Interface

- Describe changes in plain English
- Ask questions about your code
- Request explanations

4. Multi-file Support

- Understands project context
- Can work across multiple files
- Maintains relationships between files

Benefits of Using aider

For Developers:

- Faster code writing and editing
- Reduced context switching
- Built-in version control
- Natural workflow integration

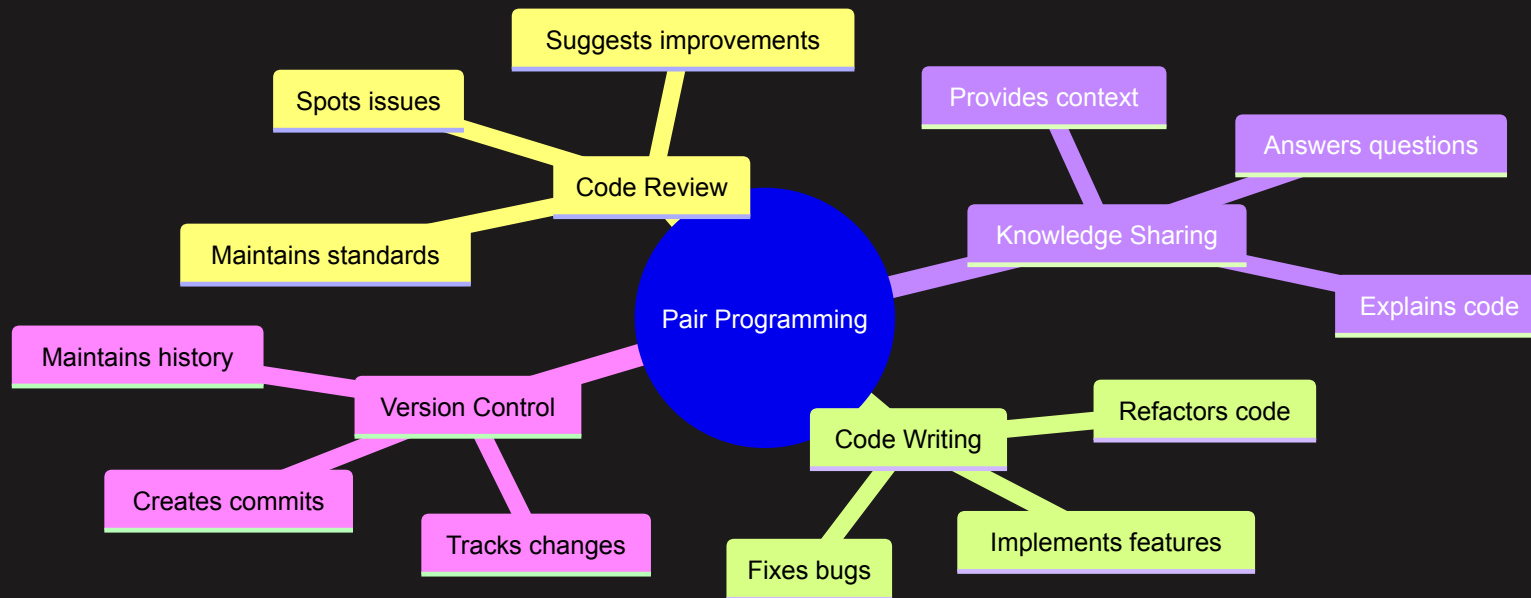
For Teams:

- Consistent code style
- Documented changes
- Easier onboarding
- Improved productivity

The Pair Programming Model

Think of aider as a pair programmer who:

- Understands your entire codebase
- Can make precise edits
- Commits changes automatically
- Is always available
- Learns your preferences



Understanding aider's Role

What aider IS:

- An AI-powered coding assistant
- A tool for accelerating development
- A pair programmer that helps with tasks

What aider is NOT:

- A replacement for human developers
- A fully autonomous coding system
- A substitute for understanding code

When to Use aider

Best Use Cases:

- Writing new code
- Refactoring existing code
- Fixing bugs
- Adding documentation
- Learning new patterns
- Exploring solutions

Less Suitable For:

- Critical security code
- Complex architectural decisions
- Business logic requiring domain expertise

Example: Basic aider Workflow

1. Start aider with your project:

```
aider path/to/project
```

2. Request a change:

```
> Add input validation to the user registration function
```

3. Review proposed changes:

```
+ def validate_user_input(username, email, password):  
+     if not username or len(username) < 3:  
+         raise ValueError("Username must be at least 3 characters")  
+     if not re.match(r"^[^@]+@[^@]+\.[^@]+", email):  
+         raise ValueError("Invalid email format")  
+     if len(password) < 8:  
+         raise ValueError("Password must be at least 8 characters")
```

4. Changes are automatically committed to Git if approved

Best Practices

1. Start Small

- Begin with simple tasks
- Learn aider's capabilities gradually
- Build confidence over time

2. Be Specific

- Give clear instructions
- Provide context when needed
- Break down complex tasks

3. Review Changes

- Always check proposed modifications
- Understand the changes
- Verify functionality

4. Use Version Control

- Commit frequently
- Keep your repository clean
- Track progress effectively

Summary

- aider is an AI-powered pair programming assistant
- It works directly with your files and Git
- Key benefits include:
 - Reduced context switching
 - Direct file editing
 - Integrated version control
 - Natural language interface
- Best used for coding tasks while maintaining developer oversight
- Follows best practices for software development